GEBZE TECHNICAL UNIVERSITY

Computer Engineering Department

CSE 396 Module Documentation

Çılgın Kurye

Module M1: Game Server and Rules

Instructor: Erkan Zergeroğlu **Company:** Board&Byte **Group:** Group-8 **Date:** 6 November 2026

Name	Student No.	Name	Student No.
Tuğberk Akbulut	230104004070	Abdullatif Yılmaz	230104004031
Berk Andaçgil	230104004066	Selim Barut	220104004046
Umutcan Ocak	220104004903		

1. Purpose

M1 is the brain of the game. It runs on the main computer and it is the only part that can change the game. It keeps the game clock and the one true game state. It knows every team, store, package, delivery point, score, fall, obstacle and earthquake. Phones only send their buttons. M1 decides what happens, and then everybody sees the same result.

Runs on	Main computer, in one server program together with M2, M3 and M4
Technology	TypeScript on Node.js; fixed-step game loop at 30 ticks per second; seeded random numbers; JSON files for the city plan and the game log
Requirements	R1 R2 R4 R5 R6 R7 R9 R11

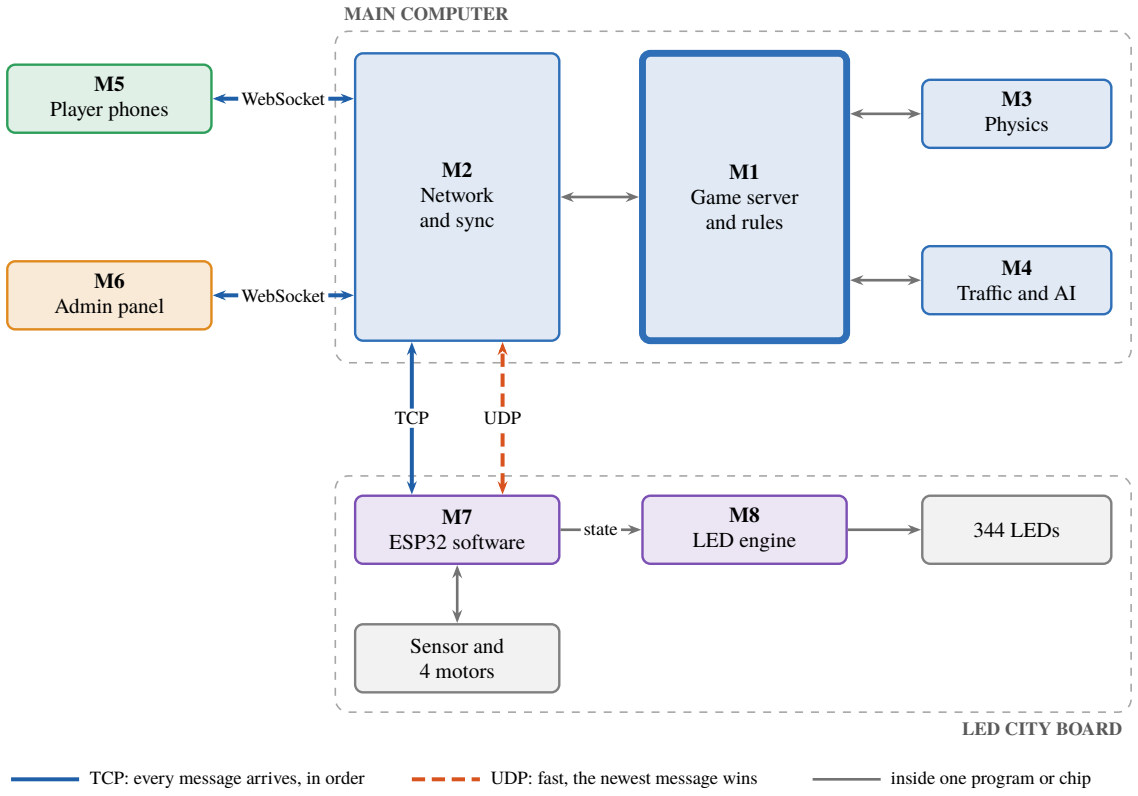


Figure 1: Where M1 sits in the system. The thick box is M1.

2. Requirements covered

ID	Requirement	What M1 does for it
R1	Multiplayer game	Lobby with 4 teams and at most 2 players in a team; bots fill empty places; a game is at most 5 minutes; the team with the most points wins
R2	Delivery cycle	Automatic pickup at the store; picks a delivery point by travel time; near or far chance per player

ID	Requirement	What M1 does for it
R4	Collisions and falls	Gives a penalty time after every fall that M3 reports
R5	No map on the phone	Puts the compass target and the “Find me” flag into the game state
R6	Obstacles	Keeps the obstacle list that M3, M4, M5 and M8 use
R7	Earthquake	One path for a hand hit, the motors and the admin button; road work and falls inside the radius
R9	Live admin panel	Applies admin commands and sends each parameter to the module that uses it
R11	Single authority	Only M1 changes the game state; every tick can be played again from the log

3. Inputs and outputs

Dir.	Data	From / to	Content
In	Controls	M2, from phones	Throttle and brake 0–1, steer −1 to 1, deliver; the newest one in each tick
In	Join, “Find me”	M2, from phones	Player key, name, team; “Find me” request
In	Admin commands	M2, from admin	Parameter changes, start, pause, end, obstacles, earthquake
In	Board events	M2, from board	Earthquake strength 0–10, centre, zone, confidence
In	Traffic	M4	NPC cars, bot controls, travel-time table
In	Physics	M3	New motorcycle positions, collision and fall events
Out	Game state	M2, to phones and admin	Every tick: phase, time, scores, players, NPC cars, obstacles, earthquake
Out	Board state	M2, to board	Players, delivery points, obstacles, earthquake in board coordinates
Out	Metrics	M2, to admin	Twice per second: the values of all admin graphs
Out	Game log	Disk	The inputs of every tick, one JSON line per tick

4. Main components

Component	What it does	Related modules
Game loop	Runs the steps of a tick in a fixed order, 30 times per second	M3, M4
Match and lobby	Game phases, teams, bots, keeps a place for 30 s after a player drops	M2, M4
Delivery and scoring	Pickup, delivery check with a reason when it says no, choice of the delivery point, points	M4, M5, M8
Penalties	Penalty time after a fall; asks M3 to stand the motorcycle up again	M3
Obstacles and earthquake	Obstacle list with a version number; earthquake strength, radius, road work and falls	M3, M4, M7
Admin and parameters	Checks commands, keeps every parameter with its limits, sends each group to its owner	M2, M3, M4, M6
State builder and recorder	Builds the frozen game state, the board state and the metrics; writes and replays the game log	M2

5. How it works

The tick. The game moves in small steps called ticks. There are 30 ticks in one second, so one tick has about 33 ms. A tick always runs the same eight steps in the same order, as the figure below shows. First M1 takes the inputs that M2 has collected since the last tick. If a phone sent more than one control message, only the newest one is used. Then M1 applies admin commands and board events, updates the game phase, asks M4 for traffic and bot controls, and asks M3 to move all motorcycles. After that the game rules run. At the end M1 builds a frozen copy of the state for M2 and writes one line to the game log. Because the order never changes and all random numbers come from one seed, the same inputs always give the same game.

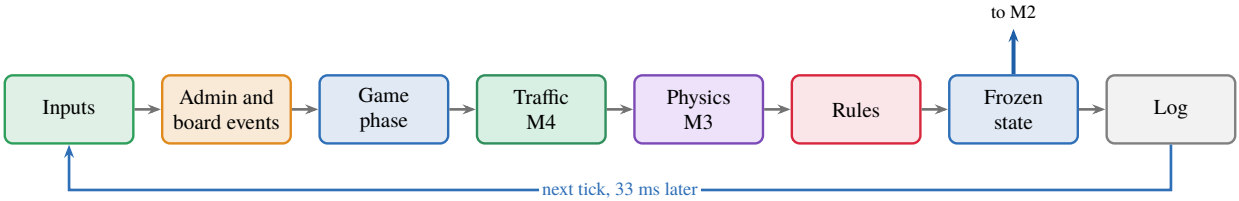


Figure 2: The eight steps of one tick.

Game flow. A game has five phases. In the lobby players join and choose a team. When the admin presses start, a 3 second countdown begins and nobody can move. Then the game runs for at most 5 minutes. The admin can pause and resume the game at any time. When the time ends, M1 shows the final scores and the game is finished. If a phone loses its connection, M1 keeps that player’s place for 30 seconds. If the player does not come back, a bot takes the motorcycle and continues.

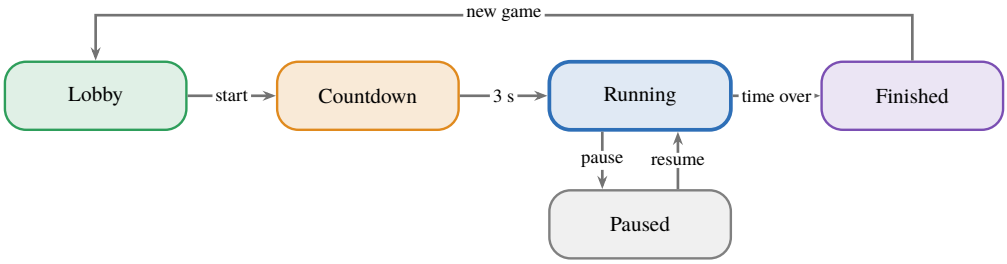
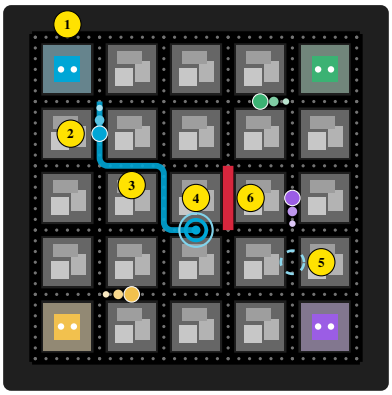


Figure 3: Phases of one game.

Deliveries and points. A player gets a package automatically when the motorcycle is inside the area of the own team's store. M1 then picks a delivery point for this player. M4 gives a table with the travel time from the store to every possible point. The times also count the obstacles that are on the roads at that moment. M1 sorts this list and cuts it in the middle, so it gets a near half and a far half. The admin sets a "near chance" for each player, and M1 uses it to choose the half. A far point gives more points: points are a base value plus a bonus for each second of the expected travel time. Both numbers are admin parameters. To deliver, the player presses "Deliver". The button is always active. M1 accepts it only inside the delivery radius and below the speed limit. If M1 says no, it sends the reason, for example "too fast" or "too far". A fall does not lose the package; the player only waits for the penalty time.



1 Team store, two LEDs in team colour
2 Player with a package, dot with a tail
3 Fastest route found by M4
4 Near delivery point, blinks in team colour
5 A far point, worth more points
6 Road work, so the route goes around

Figure 4: One delivery as it will look on the LED board. The phone shows only a compass arrow.

Obstacles and earthquakes. M1 keeps one list of obstacles. Each obstacle has a type, a road segment, a lane and a position. The list has a version number, and the number goes up on every change. So M4 can see when it must compute routes again, and M5 and M8 can see when they must draw again. All earthquakes take one path in M1, so a hand hit on the board, the motors and the admin button act in the same way. The game strength is the measured strength times an admin factor. Roads inside the earthquake radius get road work, and motorcycles inside the radius may fall.

Admin parameters. Every parameter has a name, a group, a minimum, a maximum and an owner module. When the admin changes a value, M1 checks the limits first. Then it saves the value, sends it to the owner (for example grip to M3 or traffic density to M4) and sends "ok" back to the panel. A wrong value gets an error and the old value stays. Twice per second M1 also puts all graph values (scores, delivery times, speeds, falls, traffic) into one metrics message.

6. Testing and demonstration

M1 has no network code. So we can test every rule with fake M2, M3 and M4 parts that run inside the test. The tests check the game flow, the lobby and the bots, the delivery cycle, the fair choice of points, every admin command and the earthquake. One test plays a recorded game again with the same seed and checks that the result is the same. In the demonstration, a full game with bots runs on the main computer. The admin changes parameters live, starts an earthquake, and then plays the game log again.